

Lab 05: A “gentle” Introduction to Decision Trees in R

Andrés Oswaldo Calderón Romero, PhD.

October 1, 2026

1 Introduction

Decision trees are a type of Supervised Machine Learning where the data is continuously split according to specific parameters. They are highly interpretable, mimicking human decision-making processes.

2 The Play_Tennis Dataset

We are using the [play_tennis_dataset.csv](#). This dataset predicts whether to play tennis based on the **Outlook**, **Temperature**, **Humidity**, and **Wind**.

2.1 Building the Model in R

The `rpart` package is the standard tool for creating Recursive Partitioning trees in R.

```
# Load necessary libraries
library(rpart)
library(rpart.plot)

# Load the dataset
tennis_data <- read.csv("play_tennis_dataset.csv")

# Remove the 'Day' column (unnecessary for prediction)
tennis_data$Day <- NULL

# Build the model with custom controls to force depth
model <- rpart(PlayTennis ~ .,
               data = tennis_data,
               method = "class",
               control = rpart.control(cp = 0,           # No minimum improvement
                                       minsplit = 2,      # Split even with 2 samples
                                       maxdepth = 2))    # Limit to 2 levels deep

# View the basic structure
print(model)
```

Listing 1: Model construction using `rpart`.

2.2 Mathematical Foundation: Entropy

Decision trees split the data at the point that maximizes “purity”. This is often measured using **Entropy**.

Play Tennis Decision Logic

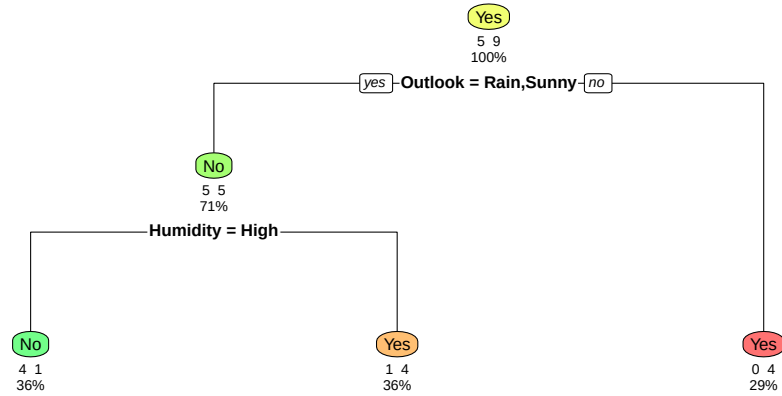


Figure 1: Play tennis classification tree.

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

By calculating the Entropy before and after a split, we determine the **Information Gain**. The variable with the highest Information Gain becomes the node.

2.3 Visualizing the Results

The `rpart.plot` package provides a “ready-to-publish” visualization of the tree logic. We will get a tree as shown figure 1.

```
# Visualize the decision-making process
rpart.plot(model,
  type = 2,
  extra = 101,
  under = TRUE,
  box.palette = "GnYlRd",
  main = "Play Tennis: 2-Level Decision Logic")
```

Listing 2: Visualizing the decision tree.

2.4 Predicting New Data

Finally, we can pass new observations to our model to get a “Yes” or “No” prediction.

```
# Create a new weather scenario
new_weather <- data.frame(
  Outlook = "Rain",
  Temperature = "Mild",
  Humidity = "High",
  Wind = "Weak"
)

# Generate prediction. Output should be: 'Prediction for the new day: No'
res <- predict(model, new_weather, type = "class")
cat("Prediction for the new day:", as.character(res))
```

Listing 3: Predicting with the trained model.

2.5 Summary of Parameters

- **Level 0 (Root):** Usually splits on the most important variable (e.g., Outlook).
- **Level 1:** Secondary decisions based on the remaining entropy (e.g., Humidity or Wind).
- **Overfitting Note:** While forcing a tree to be deeper makes it look better, be careful! On small datasets, a deep tree might learn “noise” rather than actual patterns.

3 The Advertisement Dataset

We will build a classifier to predict if a person purchases a product based on their *Age* and *Estimated Salary* using the `rpart` package. This exercise was taken at “*Decision Tree Classifiers in R Programming*” tutorial from [GeeksforGeeks](https://www.geeksforgeeks.org/r-language/decision-tree-classifiers-in-r-programming/)¹ portal.

3.1 Installing and Loading Packages

Load the necessary libraries for model construction, data splitting, and visualization.

```
# run the below line just one time...
# install.packages(c("rpart", "rpart.plot", "caTools", "ggplot2", "dplyr"))

library(rpart)
library(rpart.plot)
library(caTools)
library(ggplot2)
library(dplyr)
```

3.2 Loading the Dataset

The dataset contains advertising expenditures across media channels and their impact on sales. You can download the dataset [here](https://www.geeksforgeeks.org/r-language/decision-tree-classifiers-in-r-programming/).

```
# Loading the Advertisement dataset
dataset <- read.csv("Advertisement.csv")
```

¹@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved. 2026. “*Decision Tree Classifiers in R Programming*”. Available at: <https://www.geeksforgeeks.org/r-language/decision-tree-classifiers-in-r-programming/>. Accessed: April 2026.

```
head(dataset, 10)
```

	User.ID	Gender	Age	EstimatedSalary	Purchased
1	15624510	Male	19	19000	0
2	15810944	Male	35	20000	0
3	15668575	Female	26	43000	0
4	15603246	Female	27	57000	0
5	15804002	Male	19	76000	0
6	15728773	Male	27	58000	0
7	15598044	Female	27	84000	0
8	15694829	Female	32	150000	1
9	15600575	Male	25	33000	0
10	15727311	Female	35	65000	0

3.3 Preparing the Data

Encode the target variable as a factor and split the data into training (75%) and testing (25%) sets.

```
# Convert target variable to factor
dataset$Purchased <- factor(dataset$Purchased, levels = c(0, 1))

# Splitting dataset
set.seed(42)
split <- sample.split(dataset$Purchased, SplitRatio = 0.75)
training_set <- subset(dataset, split == TRUE)
test_set <- subset(dataset, split == FALSE)
```

3.4 Feature Scaling

Normalize numerical features so that they share the same scale.

```
# Normalize numerical features
training_set[c("Age", "EstimatedSalary")] <- scale(training_set[c("Age",
↪ "EstimatedSalary")])
test_set[c("Age", "EstimatedSalary")] <- scale(test_set[c("Age",
↪ "EstimatedSalary")])
```

```
head(training_set, 5)
```

	User.ID	Gender	Age	EstimatedSalary	Purchased
3	15668575	Female	-1.1010703	-0.8561205	0
5	15804002	Male	-1.7902649	0.1299869	0
6	15728773	Male	-1.0026140	-0.4078899	0
7	15598044	Female	-1.0026140	0.3690432	0
8	15694829	Female	-0.5103322	2.3412580	1

```
head(test_set, 5)
```

	User.ID	Gender	Age	EstimatedSalary	Purchased
1	15624510	Male	-1.7695949	-1.268692	0
2	15810944	Male	-0.3588566	-1.240511	0
4	15603246	Female	-1.0642257	-0.197828	0
14	15704987	Male	-0.6233700	-1.296872	0
22	15736760	Female	0.6991972	-0.423273	1

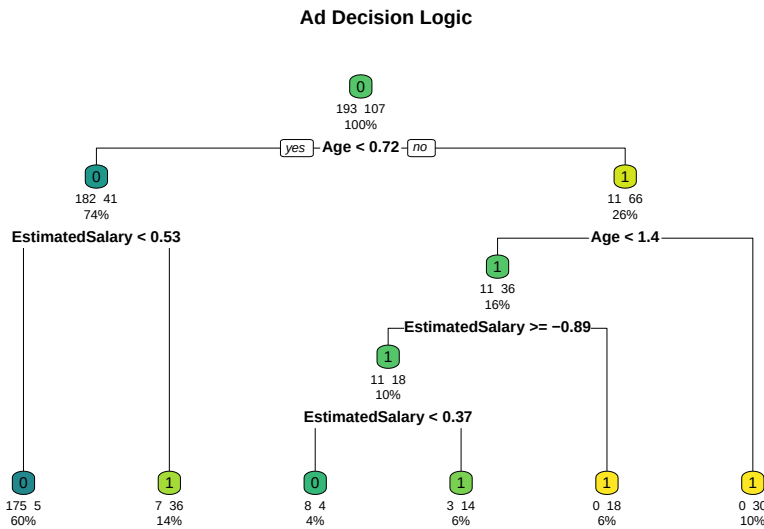


Figure 2: Ad classification tree.

3.5 Fitting the Classifier

Train the recursive partitioning model on the training data.

```
# Fitting Decision Tree to the Training set
classifier <- rpart(formula = Purchased ~ Age + EstimatedSalary,
  data = training_set,
  method = "class")
```

3.6 Visualization

Visualize how the decision tree partitions the predictor space. Result appears in figure 2.

```
# Decision tree
rpart.plot(classifier,
  type = 2,
  extra = 101,
  under = TRUE,
  box.palette = "BlGnYl",
  main = "Ad Decision Logic")
```

3.7 Model Evaluation

After fitting the model, it is essential to measure its performance using a **Confusion Matrix** to calculate metrics like accuracy.

3.7.1 Predicting Test Set Results

Apply the model to the independent test set to generate predictions.

```
# Predicting the Test set results
y_pred <- predict(classifier, newdata = test_set[-5], type = "class")
```

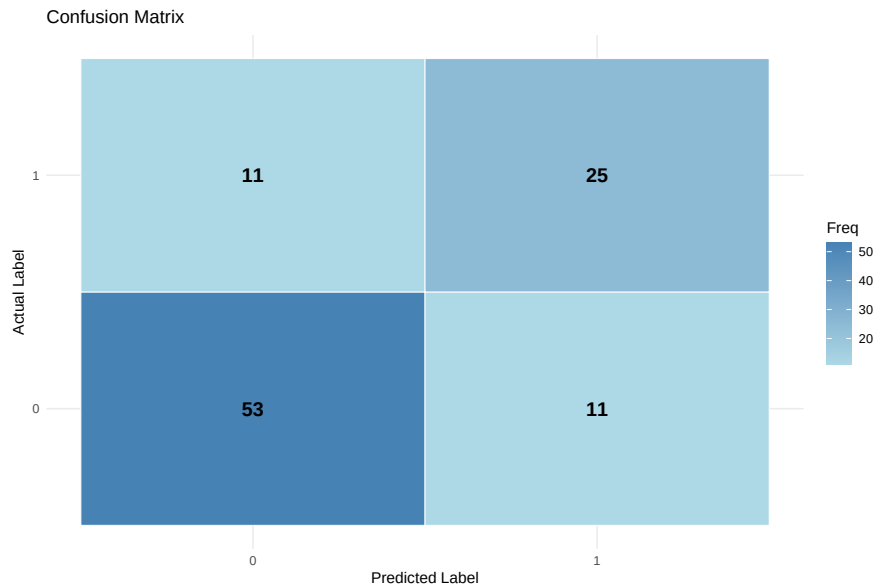


Figure 3: Decision matrix.

3.7.2 Confusion Matrix and Accuracy

Compare the predicted classes against actual values. Figure 3 shows the results.

```
# Creating the Confusion Matrix
cm <- table(test_set[, 5], y_pred)
print(cm)

# Accuracy calculation
accuracy <- sum(diag(cm)) / sum(cm)
print(paste("Model Accuracy: ", round(accuracy, 2)))
```

4 Autonomous Work

In this section, you are required to replicate the decision tree implementation detailed in Section 3 using an alternative programming ecosystem. The goal is to demonstrate that while syntax varies, the underlying machine learning logic remains consistent across platforms.

You may select one of the following language-library combinations:

Python scikit-learn or XGBoost / CatBoost

Java Smile or Weka

Julia DecisionTree.jl

C++ MLPACK or Dlib

JavaScript / TypeScript: ml-cart or TensorFlow.js

Rust SmartCore or Linfa